

Multilingual Groundedness Verification with LLM Judges: A Planted-Error Benchmark in Eleven Languages

Salim Mustafazada

SIBA — "Süni İntellekt Biznes Avtomatlaşdırma" MMC, Baku, Azerbaijan · salim@siba.az · September 2026

Abstract. Retrieval-augmented assistants fail their users most often not by looping or inventing history but by stating a wrong price, hour or address that the documents they were given do not support. Every published open-weights detector for this failure (Vectara HHEM, LettuceDetect, Patronus Lynx) is trained on English. We describe *groundedness*, a dependency-free open-source package that checks an answer against its sources by asking a general-purpose language model to list the unsupported claims and rewrite the answer without them, in the answer's own language, over any OpenAI-compatible endpoint. We introduce a small planted-error benchmark: one business's facts, one grounded answer and two answers with a single planted error (a wrong price, an invented opening day) in eleven languages (English, Azerbaijani, Russian, Turkish, Ukrainian, Kazakh, Arabic, Persian, Hindi, Indonesian, Vietnamese). Seven of nine models tested catch all 22 planted errors with zero false alarms in every language, including Kazakh, Persian and Vietnamese; a 27B open model does so with a median latency of 0.27 s. A 7B Arabic-centred model flags every grounded answer and misses a third of the errors. We also report a measurement pitfall: thinking models given a small token budget return empty replies, and a scorer that treats an empty reply as "grounded" produces a perfect-looking zero. Code, data and results are at github.com/aghasalim/groundedness under the MIT licence.

1. Problem

A business assistant that answers from the owner's documents has one failure that matters commercially: it says something the documents do not say. The Vectara hallucination leaderboard measures a related quantity, unsupported content in summaries, for roughly 150 models, in English. The open-weights detectors built for retrieval-augmented generation — HHEM-2.1 (a cross-encoder), LettuceDetect (a token classifier on RAGTruth) and Lynx (a Llama-3 fine-tune) — are likewise English-only, and Lynx is licensed for non-commercial use. A developer building for Azerbaijani, Kazakh or Vietnamese users has no detector at all.

The question this report asks is narrow: does the obvious alternative — asking a capable general model to act as the judge, in the language of the answer — actually work outside English, and which models can be trusted to do it?

2. Method

The judge. A single chat completion. The system prompt carries the sources as numbered facts and instructs the model to list every factual claim in the answer that the facts do not support (a price, a date, a time, an address, a phone, a name, an availability, a rule, a number, a quote), to treat greetings and offers to help as non-claims, and to rewrite the answer in the same language and tone with the unsupported claims removed, replying in JSON with fields *unsupported* and *answer*. Temperature 0. The package parses the JSON, strips code fences, and returns the unsupported list, the rewrite and a score (one minus the share of the answer's characters the unsupported claims occupy). An empty or non-JSON reply is returned as *judged* = *false* and is never treated as grounded (Section 4).

The benchmark. One scenario, a dental clinic: address, opening days and hours, consultation price, phone. For each of eleven languages a native-form set was written by hand: the facts; a grounded answer that restates them; an answer with the price changed from 30 to 25; and an answer that claims Sunday opening when the facts say Monday to Saturday. A planted error counts as caught when the judge reports the answer as not grounded *and* the planted text appears in the unsupported list; a grounded answer counts as a false alarm when it is reported as not grounded. Eleven languages $\times$ 3 answers = 33 calls per model.

Models. Every chat model available on Groq's free tier on 19 September 2026 (openai/gpt-oss-120b, openai/gpt-oss-20b, qwen/qwen3.8-27b, allam-2-7b) and Google's current Gemini line over its OpenAI-compatible endpoint (gemini-2.5-flash, gemini-2.5-pro, gemini-3.5-flash, gemini-3.8-flash). Calls were paced and retried on rate limits; latency is the median wall-clock time per call from a laptop in Baku.

3. Results

Model	Provider	Errors caught	False alarms	Languages perfect	Median latency
qwen/qwen3.8-27b	Groq	22/22	0/11	11/11	0.27 s
openai/gpt-oss-20b	Groq	22/22	0/11	11/11	0.48 s
openai/gpt-oss-120b	Groq	22/22	0/11	11/11	0.58 s
allam-2-7b	Groq	15/22	11/11	4/11	0.25 s
gemini-3.8-flash	Google	22/22	0/11	11/11	2.66 s
gemini-2.5-flash	Google	22/22	0/11	11/11	4.25 s
gemini-3.5-flash	Google	22/22	0/11	11/11	4.40 s
gemini-2.5-pro	Google	21/21	0/11	10/11	10.60 s

Table 1. Planted errors caught out of 22 (11 languages $\times$ 2), grounded answers wrongly flagged out of 11, languages in which both planted errors were caught, and median latency per call. One gemini-2.5-pro call (Azerbaijani, wrong price) returned a transient request error and is excluded rather than guessed, hence 21.

Seven models are perfect: every planted error caught, no grounded answer flagged, in all eleven languages. The scripts span Latin (with Azerbaijani, Turkish and Vietnamese diacritics), Cyrillic (Russian, Ukrainian, Kazakh), Arabic and Perso-Arabic, and Devanagari; none of them changed the outcome for these models. The fastest perfect judge, qwen/qwen3.8-27b on Groq, answers in a median 0.27 s, an order of magnitude faster than the Gemini Flash models and forty times faster than gemini-2.5-pro, at no loss of recall on this set.

The one clear failure is allam-2-7b, a 7B model tuned for Arabic: it reports every one of the 11 grounded answers as unsupported and misses 7 of 22 planted errors. It is the only model here below roughly 20B parameters, and the pattern — over-flagging plus misses — is what an instruction-following limit looks like, not a language limit: it fails in English too.

4. A measurement pitfall

The first Gemini run produced a table in which gemini-2.5-pro caught 0 of 22 errors with 0 false alarms and gemini-2.5-flash caught 7. Both numbers were artefacts. The judge was called with a 1,200-token output budget; these models reason before their first visible character and had spent the budget on reasoning, returning an empty *content* field. An early version of the package parsed an empty reply as "no unsupported claims" and scored it as grounded. A perfectly grounded-looking model that never says anything is exactly the false result a benchmark author is most likely to publish without noticing, because it contains no false alarms.

The corrected package returns *judged* = *false* for an empty or non-JSON reply and the benchmark counts that as a miss; the output budget is 8,000 tokens. Re-run, gemini-2.5-pro caught 21 of 21. We recommend that any LLM-as-judge evaluation report the count of unjudged calls separately from misses, and never let the absence of a reply score as a pass.

5. Limitations

The benchmark is deliberately small: one domain, two error types, 33 answers per model, planted by hand by one author who is a native speaker of Azerbaijani and reads the other languages to varying degrees. It establishes that a model can read each language and follow the instruction; it does not measure fine judgement — a right number in the wrong sentence, a plausible synonym, a claim entailed but not stated. The perfect scores should be read as "not a language problem", not as "solved". Eleven languages were chosen for script diversity and for the markets the author works in; adding a language is one JSON entry. Only two providers were run, for cost; the runner takes any OpenAI-compatible endpoint and a list of model names.

The judge checks against the sources it is given, not against the world: a true claim absent from the sources is reported as unsupported. For a business assistant that must only say what its owner wrote, that is the desired behaviour; for open-domain question answering it is not the right tool.

6. Availability

The package (*pip install groundedness*; Python 3.9+, no dependencies), the eleven-language case file, the runner and both result sets are at github.com/aghasalim/groundedness under the MIT licence. The judge described here also runs on every channel of SIBA's assistant (siba.az), where each channel reports the share of answers that passed; that deployment is the origin of the method and the source of future cases.

Reproduce.

```
pip install groundedness pytest
GROQ_API_KEY=... python benchmark/run.py
GEMINI_API_KEY=... python benchmark/run.py --gemini
```

References

- Vectara. Hallucination Leaderboard. github.com/vectara/hallucination-leaderboard, 2024–2026.
- Kovács, Á. et al. LettuceDetect: A Hallucination Detection Framework for RAG Applications. arXiv:2502.17125, 2025.
- Ravi, S. et al. Lynx: An Open Source Hallucination Evaluation Model. arXiv:2407.08488, 2024.
- Manakul, P., Liusie, A., Gales, M. SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. EMNLP 2023.
- Farquhar, S. et al. Detecting hallucinations in large language models using semantic entropy. Nature 630, 2024.
- Aghayev, F., Ahmadbayli, E. SIMURG: Zero-Leak Online Detection of LLM Decoding Corruption in Production Streams. HAL-X AI, 2026.